
An Agentic AI Science Community for Automated Neural Operator Discovery

Luis Loo

Department of Electrical & Computer Engineering
Texas A&M University, College Station, TX
luis.loo@tamu.edu

Ulisses Braga-Neto

Department of Electrical & Computer Engineering
Texas A&M University, College Station, TX
ulisses@tamu.edu

Abstract

Neural operators such as Fourier Neural Operators (FNO) and DeepONets carry fixed inductive biases, and choosing the right family and architecture for a given partial differential equation (PDE) remains a manual, expertise-driven task. We present an *agentic AI Science Community* for automated neural operator discovery: a swarm of virtual laboratories that collaborate under a citation-based economy of influence to discover operator architectures with no human prior on architecture. Each lab couples three role agents—a *planner* that proposes an architecture, a *worker* that trains and measures it, and a *reviewer* that participates in cross-lab peer review—over a shared block vocabulary (Fourier, attention, wavelet, residual convolution, branch–trunk). The planner and reviewer are driven by a large language model (LLM); the worker is always numerical (gradient-based training), by design, so the fitness signal stays comparable across the swarm. We evaluate on five PDE regimes (piecewise regression, linear advection, Burgers in 1D; Navier–Stokes and Darcy in 2D), three seeds each, and we *ablate* the LLM agents against a rule-based particle-swarm coordinator to isolate what language-model agency contributes. All 9,623 LLM calls are logged and audited, making the claim “the agents are LLMs” verifiable rather than assumed: we observe zero rule-based fallbacks. The ablation yields a mechanistic finding: at equal training budget the two coordinators reach statistically indistinguishable screening accuracy, but they discover systematically *different designs*. The analytic swarm collapses to canonical single-family answers (pure-Fourier stacks on Navier–Stokes, wavelet stacks on Darcy), whereas the LLM planner—which chose to hybridize in 99.8% of its logged decisions—consistently returns multi-family hybrids that match this accuracy at lower parameter cost (e.g. $2.5\times$ fewer parameters on Navier–Stokes), and it rediscovers the same four-block advection recipe in independent seeds. Converged cross-method baselines confirm the honest headline, matching the operator-learning literature: there is *no universal winner*—FNO dominates the periodic Navier–Stokes regime while DeepONet-style models dominate the elliptic Darcy regime—and the converged winners track it: on Darcy the discovered hybrid attains the best mean error in our baseline suite, while on Navier–Stokes canonical pure-Fourier stacks remain unbeaten—the rule-based arm recovers exactly that answer, and the LLM’s hybrid trails it at a matched parameter budget. Each winner degrades sharply on the other regime. We further quantify the wall-clock cost of LLM agency. Our contribution is methodological: reproducible, auditable agentic discovery of competitive hybrid

operators. Code, configurations, and the complete LLM transcripts are released at <https://github.com/luislootx/AI-SC>.

1 Introduction

Operator learning has become a central tool for building fast surrogates of partial differential equations (PDEs) [11, 14, 8]. Yet each operator family encodes a fixed inductive bias: global spectral convolutions for the Fourier Neural Operator (FNO) [11], a branch–trunk decomposition for DeepONet [14], multiresolution bases for wavelet operators [24], and attention for transformer-style operators [2, 5]. Selecting the family, depth, width, and block composition for a new PDE is still done by hand, and—as careful head-to-head studies show—no single family is best across problem regimes [15]. This raises a natural question:

Can a community of automated agents discover the right neural operator architecture for a given PDE, with no human prior on architecture?

We answer this by instantiating the *AI Scientific Community* framework [1]—a swarm of virtual laboratories that explore a design space under a citation-based economy, where well-regarded labs gain influence and breeding priority while uncited labs are culled and replaced—for neural operator discovery. Crucially, we drive the *planning* and *peer-review* roles of each lab with a large language model, while keeping the *training* role numerical so that the fitness signal remains comparable across labs and iterations. This makes the contribution of language-model agency *measurable*: we ablate the LLM planner/reviewer against an analytic particle-swarm coordinator under an otherwise identical protocol.

Contributions.

1. **Agentic operator discovery.** An instantiation of the AI Scientific Community for neural operator architecture search, with explicit LLM planner and reviewer agents and a deterministic numerical worker (Section 4–5).
2. **A controlled ablation of LLM agency.** LLM-agent vs. rule-based particle-swarm coordination across five PDE regimes and three seeds, isolating what the language model adds under an identical training budget. The answer is architectural rather than accuracy: the LLM returns parameter-lean multi-family hybrids where the analytic swarm collapses to canonical families (Section 7).
3. **An evidence-first methodology.** Every LLM call (9,623 across the campaign) is logged to a transcript; we report per-run call counts and fallback rates, so “the agents are LLMs” is auditable, not assumed (Section 5.3, Table 3).
4. **A computational-cost analysis.** We quantify per-iteration and per-lab wall-clock cost, isolating the overhead of LLM agency and situating it against single-operator training cost at screening fidelity (Section 8).
5. **An honest cross-regime result.** No operator is universal; the swarm recovers regime-appropriate designs—on Darcy, a hybrid attaining the best mean converged error in our baseline suite; on Navier–Stokes, the canonical pure-Fourier answer via the rule-based arm—and maintains population diversity rather than collapsing to a single recipe (Section 7).

2 Related Work

Neural operators. FNO [11] learns a kernel in Fourier space; DeepONet [14] separates input-function encoding (branch) from query-location encoding (trunk); the neural operator framework unifies these as learnable maps between function spaces [8]. The design space has since broadened considerably: wavelet operators [24] for multiresolution bases, transformer/attention operators [2, 5], U-shaped multiscale operators [17], and geometry-adapted variants for irregular domains [10]. Physics-informed formulations [18] and standardized benchmarks [22] support systematic comparison. A key empirical lesson, central to our framing, is that the best operator is problem-dependent: rigorous comparisons find FNO and DeepONet each win in different regimes [15]. The breadth of this

space, and the absence of a universal winner, is precisely what motivates an automated, per-problem search over operator blocks.

Architecture search. Neural architecture search (NAS) automates model design via reinforcement learning [29], evolutionary search [19], gradient-based relaxation of the search space [12], and many other strategies [4]. A more radical line evolves entire learning algorithms from primitive operations rather than selecting among predefined blocks [20]. Our swarm of labs is an evolutionary/particle-swarm [7] search over an operator-block genome; the novelty here is not the search operator but that the proposal and selection policies can be delegated to a language model and ablated against the analytic baseline.

LLMs as optimizers and architecture proposers. Large language models have been used as black-box optimizers [26], as proposal engines for neural architecture search [28], and for program- and hypothesis-search in mathematical and scientific discovery [21]. Closest to our planner is EvoPrompting [3], which uses a language model as the mutation and crossover operator inside an evolutionary loop to propose code-level architectures. Our planner plays the same role over an operator-block genome, but we make two design choices that sharpen measurability: the fitness signal comes from a deterministic training procedure rather than a learned predictor, and every LLM-driven run is paired with a rule-based particle-swarm [7] arm so the language model’s contribution is isolable by ablation.

Multi-agent LLM systems. A growing body of work composes multiple LLM agents with distinct roles that communicate to solve a task: conversational multi-agent frameworks [25], communicative agent societies [9], and role-specialized teams that emulate a software company [16] or a standardized-operating-procedure organization [6]. At the workflow level such teams drive end-to-end research agents [13] and numerical algorithm design [23]. Our framework instantiates this paradigm with a planner–worker–reviewer triad per lab; the reviewer’s accept/reject vote is a form of LLM-as-judge evaluation [27], here applied to peer review of candidate operators rather than to chatbot responses. We build directly on the AI Scientific Community model of agentic virtual-lab swarms [1], and contribute a controlled, ablation-backed study of when the language-model agents actually outperform their rule-based counterparts.

3 Problem Formulation

Operator learning. Given a PDE with solution operator $\mathcal{G} : \mathcal{A} \rightarrow \mathcal{U}$ mapping an input function $a \in \mathcal{A}$ (e.g. an initial condition or coefficient field) to the solution $u = \mathcal{G}(a) \in \mathcal{U}$, we seek a parametric surrogate $\mathcal{G}_\theta$ minimizing the expected relative L_2 error

$$\mathcal{L}(\theta) = \mathbb{E}_{a \sim \mu} \frac{\|\mathcal{G}_\theta(a) - \mathcal{G}(a)\|_2}{\|\mathcal{G}(a)\|_2}, \quad (1)$$

estimated on sampled input–output pairs. The relative L_2 error is our primary accuracy metric throughout.

Architecture search space. We define an architecture by a *genome* $g = (b_{1:L}, c, m, \sigma, \dots)$, where $b_{1:L}$ is a sequence of $L \in [2, 8]$ blocks drawn from a vocabulary $\mathcal{B} = \{\text{FOURIER}, \text{ATTENTION}, \text{WAVELET}, \text{RESIDUAL_CONV}, \text{BRANCH_TRUNK}\}$, c is the hidden width, m the number of spectral modes, σ the activation, plus gating, skip connections, dropout, and learning rate. A genome compiles to a concrete neural operator $\mathcal{G}_{\theta(g)}$. The discovery problem is the bilevel program

$$g^* = \arg \min_g \mathcal{L}(\theta^*(g)), \quad \theta^*(g) = \arg \min_\theta \hat{\mathcal{L}}_{\text{train}}(\theta; g), \quad (2)$$

where the inner problem is ordinary gradient-based training of a fixed architecture and the outer problem searches genomes. We solve the outer problem with a swarm of agentic labs (Section 4); the inner problem is always numerical.

Algorithm 1 AI Scientific Community swarm (one configuration)

```
1: initialize  $N$  labs across paradigms (FNO, DeepONet, transformer, wavelet, random/hybrid)
2: for iteration  $t = 1, \dots, T$  do
3:   for each active lab  $i$  do ▷ Phase 1: plan
4:      $g_i \leftarrow \text{PLAN}(g_i, g^{\text{best}}, \{g_j\}_{j \neq i}, \text{history}_i)$  ▷ LLM agent or PSO update
5:   end for
6:   for each active lab  $i$  do ▷ Phase 2: train (always numerical)
7:      $\theta_i \leftarrow \text{TRAIN}(g_i)$ ; measure relative  $L_2$ 
8:   end for
9:    $\{v_i\} \leftarrow \text{PEERREVIEW}(\{(g_i, \text{fitness}_i)\})$  ▷ Phase 3: LLM agent or accuracy-proportional
10:  update composite fitness, trust, global best; cull uncited, breed influential ▷ Phase 4
11:  checkpoint state ▷ resume-safe
12: end for
```

4 The AI Scientific Community Framework

A swarm of N virtual labs explores the genome space. Each lab i holds a current genome g_i , a personal best, a velocity (for the analytic update), and a *trust* score accumulated through peer review. Each iteration runs four phases (Algorithm 1, Figure 1):

1. **Planning.** Each lab proposes its next genome g_i .
2. **Training.** Each lab compiles g_i and trains it at screening fidelity, returning measured relative L_2 (clean and noisy).
3. **Peer review.** Labs review a sample of peers and cast soft votes; votes accumulate as citation-like influence and update trust scores.
4. **Lifecycle.** A composite fitness combines accuracy, generalization, parameter efficiency, and novelty (with an accuracy floor); the global best is updated, influential labs persist and breed, and uncited labs are culled and replaced, realizing selection over research directions.

The composite fitness for lab i is

$$F_i = w_a A_i + w_g G_i + w_e E_i + w_n \nu_i, \quad (w_a, w_g, w_e, w_n) = (0.70, 0.20, 0.05, 0.05), \quad (3)$$

zeroed when accuracy falls below a floor, which prevents trivially small or degenerate models from winning on efficiency or novelty alone. Here A_i is an accuracy score derived from relative L_2 , G_i rewards robustness to input noise, E_i rewards parameter parsimony, and ν_i is an architectural-novelty score relative to the rest of the population. The weights are fixed a priori, shared by every run and both conditions, and were not tuned; accuracy dominates by design, and ablating the weighting itself is left to future work.

5 Method: Agentic Roles

5.1 Three role agents per lab

Each lab couples three roles. The **planner** proposes the next genome; the **worker** compiles and trains the corresponding operator with gradient descent, returning measured relative L_2 ; the **reviewer** casts peer-review votes over a sample of peer candidates. The **worker is always numerical**: we deliberately do not delegate training to a language model, so the fitness signal is consistent and comparable across labs and iterations. Thus “LLM agents” in this paper refers precisely to the *planner* and *reviewer* roles—the decision-makers—never to operator training.

5.2 LLM-agent and rule-based modes

The framework is model-agnostic in the planner/reviewer roles, which lets us run a clean ablation:

- **LLM agents.** The planner queries an LLM for the next genome, given its own history, the global best, and a summary of peer labs and their measured fitness; the reviewer queries an LLM to score candidate genomes against measured fitness. Proposals are parsed from JSON and snapped to the

The AI Scientific Community for neural operator discovery

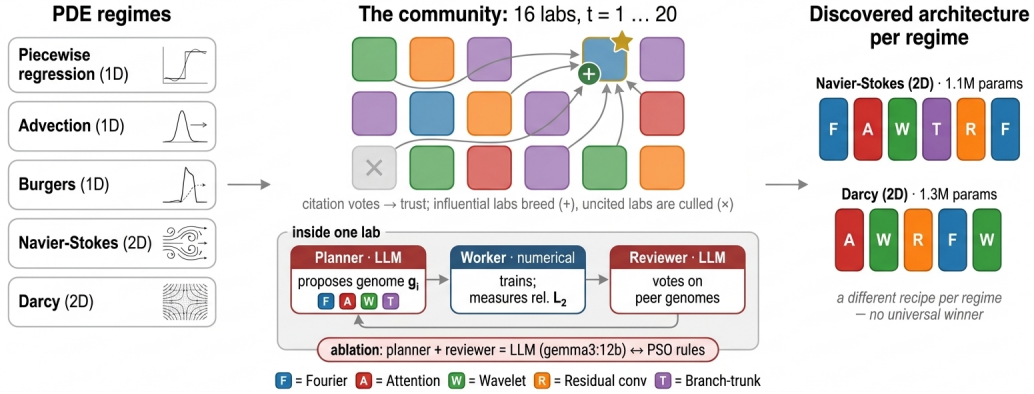


Figure 1: The agentic AI Science Community. Each of five PDE regimes (left) is attacked by a community of $N = 16$ virtual labs (center, schematic): inside every lab, a *planner* (LLM) proposes an operator-block genome g_i , a deterministic *worker* trains it (PyTorch) and measures relative L_2 , and a *reviewer* (LLM) casts soft peer-review votes over peer candidates. Votes act as citations: influential labs gain trust and breed, uncited labs are culled and replaced, over $t = 1 \dots 20$ iterations. The two decision roles are ablated against analytic PSO rules under an identical training budget. The community returns a different architecture per regime (right): the campaign’s converged winners of Table 1.

valid genome space. We serve gemma3:12b locally via Ollama, so the pipeline is reproducible with no paid API.¹

- **PSO (rules).** The planner uses analytic explore/exploit particle-swarm updates [7]; the reviewer uses accuracy-proportional voting. No language model is involved.

Both modes share swarm size, iteration count, training budget, fitness, and seeds; only the planner/reviewer policy differs. This isolates the effect of language-model agency.

5.3 Evidence that the agents are LLMs

A recurring risk in “agentic” systems is that a language model silently degrades to a rule-based fallback (e.g. on a transient error or malformed output) while still being described as an agent. We guard against this by logging *every* LLM call—system prompt, user prompt, raw response, role, and parse status—to a per-run transcript (llm_transcript.jsonl), and by auditing each run’s log for planner/reviewer fallbacks. Table 3 reports, per LLM run, the planner and reviewer call counts, the parse-success rate, and the fallback count. A fallback count of zero certifies that planning and review were LLM-driven throughout; the transcripts are released so this audit is independently reproducible.

5.4 Screening fidelity

Each candidate is trained at deliberately reduced “screening” fidelity (small data, few epochs, modest resolution) so a single operator trains in seconds, not hours, making a swarm of hundreds of trainings tractable on one GPU (Section 8). Absolute screening errors are therefore higher than fully-converged training; we report converged cross-method baselines separately to calibrate the discovered architectures.

¹We selected this model on a structured-output criterion rather than raw scale. A smaller efficiency-tier variant (gemma4:e4b) failed to emit schema-valid JSON for the reviewer role—even with prompt hardening and an enlarged context window—and silently degraded to a non-LLM fallback; a replay probe over real planner and reviewer prompts confirmed that gemma3:12b produces valid structured output for both roles where the smaller variant did not. The evidence audit (Section 5.3) exists precisely to make such silent degradation detectable, and verifies that none occurs in the reported runs.

Table 1: Converged cross-method baselines (relative L_2 , mean over seeds; parameters counted at this protocol’s resolution—block parameter counts scale with grid size). FNO wins Navier–Stokes; DeepONet-style models win Darcy. The two “Discovered hybrid” rows are the campaign’s winning architectures per regime, each retrained from scratch and evaluated on *both* regimes: each is competitive at home and degrades off-regime, which is why per-regime discovery is needed.

Model	Params	NS-2D rel. L_2	Darcy-2D rel. L_2
DeepONet faithful	603K	0.1613 ± 0.0122	0.0649 ± 0.0037
Discovered hybrid (Darcy)	1.3M	0.0018 ± 0.0002	0.0631 ± 0.0083
Discovered hybrid (NS)	1.1M	0.0005 ± 0.0000	0.7127 ± 0.0736
FNO h128 m16	33.6M	0.0002 ± 0.0000	0.3315 ± 0.0047
FNO h32 m12	1.2M	0.0002 ± 0.0000	0.3070 ± 0.0404
FNO h64 m12	4.7M	0.0002 ± 0.0000	0.3583 ± 0.0151
POD-DeepONet	142K	0.0033 ± 0.0001	0.0819 ± 0.0028
Pure Transformer	105K	0.2624 ± 0.0038	0.5207 ± 0.0102

6 Experimental Setup

We run five PDE regimes: piecewise regression, linear advection, and Burgers in 1D; Navier–Stokes (vorticity form, Kolmogorov forcing) and Darcy flow in 2D. Each configuration runs for three seeds (42, 137, 2024) in both the LLM and PSO conditions ($5 \times 2 \times 3 = 30$ runs), with $N = 16$ labs for $T = 20$ iterations (the paper-grade preset). All runs are resume-safe (checkpointed per iteration) to survive interruptions. We additionally train converged cross-method baselines (FNO at several sizes, DeepONet, POD-DeepONet, a pure transformer, and the swarm-discovered hybrid) with input/output normalization, three seeds each. All timings are on a single NVIDIA RTX 4080.

7 Results

No universal winner. Table 1 reports converged cross-method baselines on Navier–Stokes and Darcy. The pattern matches the operator-learning literature [15] and is the honest core of our story: FNO dominates the periodic, spectral-friendly Navier–Stokes regime by an order of magnitude, while DeepONet-style models dominate the elliptic, Dirichlet Darcy regime. No single operator wins both. The same table evaluates the swarm’s two discovered winners, retrained from scratch to convergence, on *both* regimes. Each is competitive with the regime-best family at home. The Darcy hybrid attains 0.0631 ± 0.0083 , the best mean Darcy error in the suite, statistically tied with DeepONet faithful (0.0649 ± 0.0037) at three seeds. The NS hybrid reaches 0.0005 at 1.1M parameters—an order of magnitude below every non-FNO baseline and $30\times$ fewer parameters than the large FNO—but we note plainly that a canonical pure-Fourier stack at the *same* budget is better still (FNO h32: 0.0002 at 1.2M), and that this canonical stack is exactly what the PSO arm discovers on NS. Off-regime, each winner degrades: the NS hybrid collapses on Darcy (0.71) and the Darcy hybrid gives up $3.6\times$ on NS (0.0018). The pattern is symmetric and honest: rule-based search wins precisely where the textbook answer is optimal, LLM hybridization pays where no single family suffices, and neither transfers—exactly the no-free-lunch behavior that motivates discovering a different architecture per regime rather than reusing one.

Agentic discovery: hybrids per regime vs. canonical families. Table 2 reports, per PDE, the relative L_2 and parameter count of the architecture discovered under the LLM and PSO conditions; Figure 2 visualizes the discovered block sequences, and Appendix B lists every per-seed winner. Two patterns emerge. First, *every* LLM winner is a multi-family hybrid, and the mixture is regime-appropriate: Fourier-led stacks on the 1D transport problems (on advection, two of three seeds independently discover the identical four-block recipe F-A-W-T: fourier, attention, wavelet, branch-trunk), and attention/wavelet-led mixtures on piecewise regression and Darcy. Second, the PSO condition instead collapses to canonical, nearly single-family designs: pure-Fourier stacks on Navier–Stokes (two of three seeds—the textbook answer for a periodic, spectrally friendly regime) and wavelet-dominated stacks on Darcy and piecewise regression. The trade-off against established baselines is shown in Figure 3: the discovered hybrids sit on a competitive accuracy–parameter

Table 2: Swarm-discovered architectures: relative L_2 (clean) and parameter count, LLM (Gemma 3) vs. PSO, mean $\pm$ std over three seeds.

PDE	Discovered rel. L_2 (clean)		Params	
	LLM (Gemma 3)	PSO	LLM	PSO
Piecewise Reg. (1D)	0.0417 $\pm$ 0.0011	0.0405 $\pm$ 0.0007	487K $\pm$ 258K	198K $\pm$ 95K
Advection (1D)	0.0159 $\pm$ 0.0038	0.0183 $\pm$ 0.0007	273K $\pm$ 157K	456K $\pm$ 159K
Burgers (1D)	0.0799 $\pm$ 0.0067	0.0709 $\pm$ 0.0047	472K $\pm$ 415K	197K $\pm$ 103K
Navier-Stokes (2D)	0.0009 $\pm$ 0.0005	0.0007 $\pm$ 0.0000	2.2M $\pm$ 613K	5.6M $\pm$ 4.1M
Darcy (2D)	0.0095 $\pm$ 0.0003	0.0096 $\pm$ 0.0015	1.4M $\pm$ 403K	966K $\pm$ 593K

Discovered architecture per PDE (LLM agents): different recipe per regime

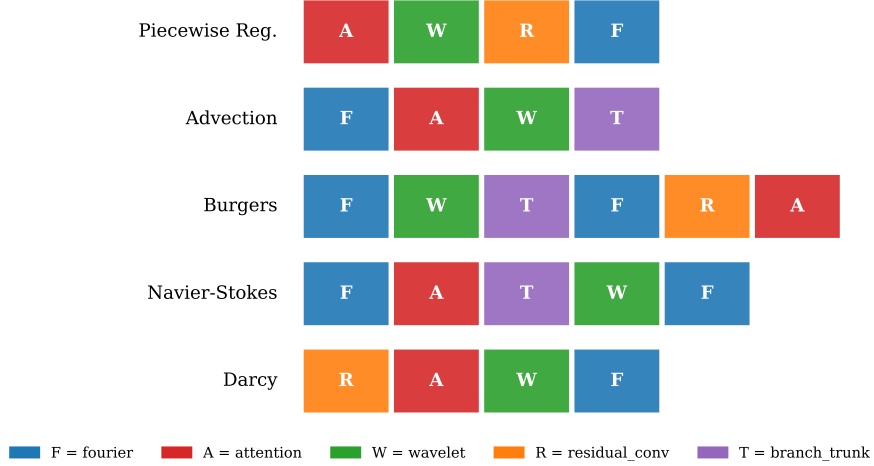


Figure 2: Discovered architecture per PDE (LLM agents, seed 42): every regime’s winner is a multi-family hybrid—Fourier-led on the 1D transport problems, attention/wavelet-led on piecewise regression and Darcy—with a different recipe per regime.

frontier; on Navier–Stokes the LLM hybrids match the pure-spectral PSO accuracy with $2.5\times$ fewer parameters on average (2.2M vs. 5.6M).

Ablation: what does LLM agency add? Figure 4 compares discovered relative L_2 under LLM and PSO coordination, and Figure 5 compares convergence of the swarm’s best error per iteration. Because both conditions train identical operators under an identical budget, any difference is attributable to the planner/reviewer policy. At three seeds the screening accuracies are statistically indistinguishable on all five PDEs (overlapping ± 1 s.d.), and we do not claim an accuracy win for LLM agency. The measurable difference is architectural: the LLM discovers parameter-lean multi-family hybrids where PSO converges to canonical families, and it does so reproducibly (the identical advection recipe across independent seeds). We report this plainly rather than claiming LLM dominance: the value of the framework is the agentic, auditable discovery process, the ablation that measures it, and the design-space behavior it reveals.

How the LLM plans. The transcripts let us inspect the planner’s *decisions*, not only its outcomes. Across 4,817 planner calls the LLM chose HYBRIDIZE in 99.8% of decisions (the remainder EXPLORE; never EXPLOIT), and its rationales condition on the summarized swarm state, e.g. “*several labs are converging on FNO and transformer architectures. . . I’ll introduce a hybrid approach by incorporating attention blocks*” (Navier–Stokes, seed 42). This action bias is the mechanism behind the architectural contrast above: a planner that almost always grafts blocks from other families structurally produces multi-family hybrids, whereas the analytic update can—and does—collapse to single-family optima. The bias is partly prompt-induced (the planner is instructed to preserve diversity when the swarm converges; Appendix A), so we read it as a caveat as much as a capability:

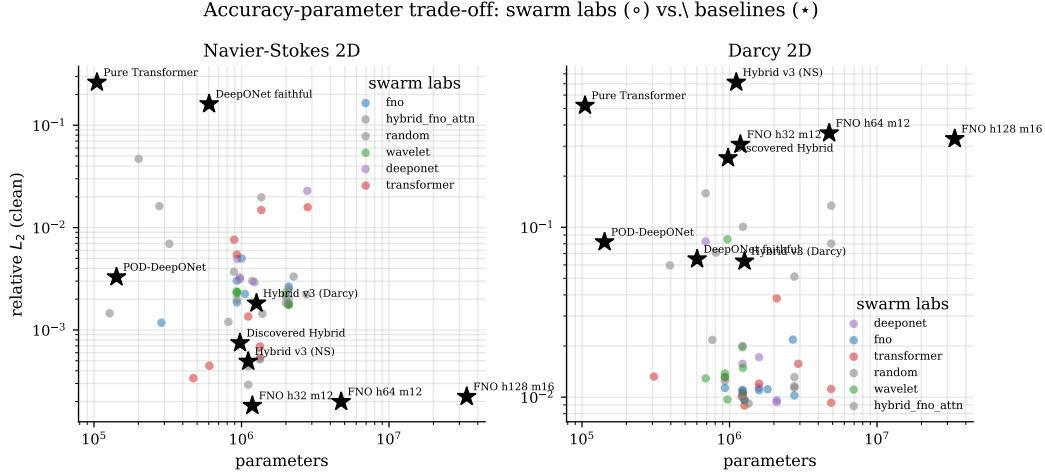


Figure 3: Accuracy–parameter trade-off on the 2D problems. Swarm labs (circles, colored by paradigm) and converged baselines (stars). The swarm populates a competitive frontier at far fewer parameters than the large FNO baselines.

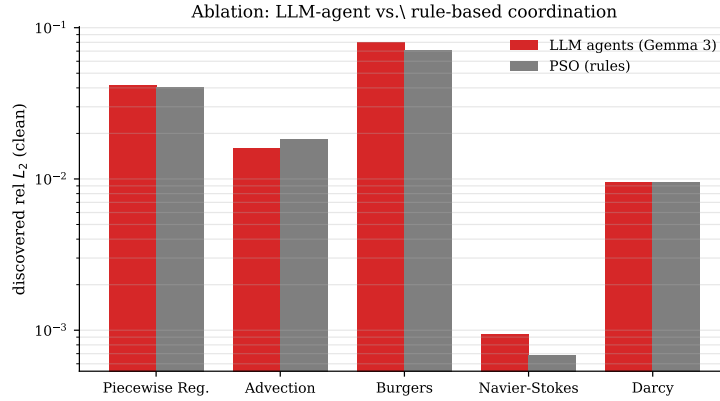


Figure 4: Discovered relative L_2 per PDE: LLM agents (Gemma 3) vs. rule-based PSO, under an identical training budget.

LLM agency here manifests as context-conditioned hybridization rather than balanced explore/exploit arbitration.

What the reviewer rewards. The same transcripts quantify the reviewer. Across 4,804 scoreable review calls, the rank correlation between the reviewer’s vote distribution and the candidates’ measured accuracy is $\bar{\rho} = 0.64$ (Spearman; positive in 88% of calls, a perfect accuracy ordering in 50%), so peer review is grounded in the fitness signal rather than stylistic preference. Its rationales cite accuracy in 97% of calls, parameter efficiency in 83%, and robustness in 71%, but novelty in only 26%: the deviations from pure accuracy ordering are consistent with the prompt’s instruction to also reward novelty and efficiency. Review is thus fitness-anchored with a diversity tilt—precisely the property the citation lifecycle needs, since accurate labs must accumulate influence without unusual designs being starved (Figure 6).

The swarm maintains diversity. A failure mode of citation-driven swarms is premature collapse onto an early leader (groupthink). Figure 6 shows population-level block usage stays mixed across iterations: the novelty term and minimum exploration keep multiple paradigms alive even as individual winners specialize. The “different recipe per regime” result (Figure 2) is thus a property of the *winner*, not a collapse of the population.

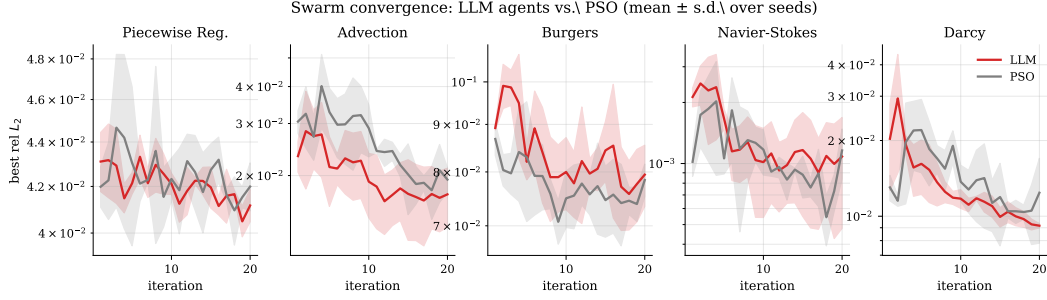


Figure 5: Swarm convergence: best relative L_2 per iteration, LLM vs. PSO, mean $\pm$ s.d. over seeds.

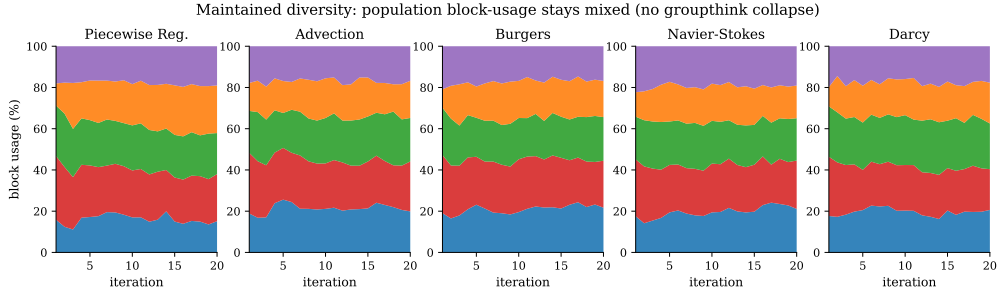


Figure 6: Population block-usage fraction across iterations (LLM). Diversity is maintained—no single block type dominates—indicating the swarm avoids groupthink collapse.

The agents really were LLMs. Table 3 summarizes the transcript audit over all LLM runs: planner and reviewer calls, parse-success, and fallbacks. Across the campaign we observe zero fallbacks, certifying that planning and review were LLM-driven.

8 Computational Cost

Table 4 and Figure 7 report wall-clock times on a single RTX 4080: the total time for a full $16\text{-lab} \times 20\text{-iteration}$ discovery run, and the time per lab-evaluation (one lab’s plan–train–review cycle), for both conditions. Because the LLM and PSO conditions train identical operators under an identical budget, the difference in per-lab time is precisely the overhead of LLM-driven planning and review (two language-model calls per lab per iteration): roughly 14s per lab-evaluation on the 1D problems and 25–33s on the 2D problems, i.e. a $2\text{--}4\times$ increase in total wall-clock over rule-based coordination. The LLM condition also carries higher run-to-run variance on the 2D problems (e.g. 271 ± 104 min on Navier–Stokes): an honest consequence of letting the planner commit to parameter-heavy hybrids whose training dominates the budget. The overhead buys the behavior of Section 7; whether that trade is worthwhile is regime- and goal-dependent, and the ablation makes it measurable.

Why a swarm of hundreds of trainings is feasible. The swarm screens architectures at reduced fidelity, so a single operator trains in seconds, not hours. Table 5 lists single-operator training times to convergence at screening fidelity on Navier–Stokes; cost is dominated by epochs and block type, not parameter count (a 33.6M-parameter FNO trains faster than a 105K-parameter transformer because attention is quadratic in sequence length). A full discovery run evaluates $16 \times 20 = 320$ architectures within the budgets above, reserving expensive full-fidelity training for the final discovered architecture and the baseline comparison—the standard multi-fidelity argument: screen cheaply, commit expensively.

Table 3: Evidence audit: per-run planner/reviewer LLM call counts, parse-success, and fallbacks. Zero fallbacks certifies LLM-driven planning and review. Counts above the nominal $16 \times 20 = 320$ reflect re-planned iterations after mid-run interruptions (the runs are checkpointed and resume-safe); every call, including repeats, is in the released transcripts.

LLM run	planner	reviewer	parse-ok	fallbacks
advec seed137	320	320	640/640	0
advec seed2024	320	320	640/640	0
advec seed42	320	320	640/640	0
burgers seed137	320	320	640/640	0
burgers seed2024	320	320	640/640	0
burgers seed42	320	320	640/640	0
darcy seed137	335	326	661/661	0
darcy seed2024	320	320	640/640	0
darcy seed42	320	320	640/640	0
ns seed137	320	320	640/640	0
ns seed2024	320	320	640/640	0
ns seed42	320	320	640/640	0
pwreg seed137	320	320	640/640	0
pwreg seed2024	322	320	642/642	0
pwreg seed42	320	320	640/640	0
total	9623 LLM calls			0

Table 4: Computational cost on a single RTX 4080: total run time and per lab-evaluation time, LLM (Gemma 3) vs. PSO, mean $\pm$ std over seeds. The LLM-PSO gap isolates the cost of language-model agency.

PDE	Total run (min)		Per lab-eval (s)	
	LLM	PSO	LLM	PSO
Piecewise Reg. (1D)	89.7 $\pm$ 18.3	26.0 $\pm$ 11.7	19.4 $\pm$ 1.8	4.9 $\pm$ 2.4
Advection (1D)	96.7 $\pm$ 10.6	29.2 $\pm$ 12.2	18.2 $\pm$ 2.2	5.5 $\pm$ 2.4
Burgers (1D)	108.3 $\pm$ 2.3	41.1 $\pm$ 0.9	19.8 $\pm$ 1.8	7.0 $\pm$ 0.5
Navier-Stokes (2D)	270.9 $\pm$ 104.3	103.1 $\pm$ 6.7	51.3 $\pm$ 27.1	18.7 $\pm$ 3.3
Darcy (2D)	173.6 $\pm$ 71.9	99.9 $\pm$ 7.9	42.1 $\pm$ 15.9	18.4 $\pm$ 2.6

9 Discussion and Limitations

LLM-as-policy, not autonomous tool use. Our agents use the LLM as a *policy* for the planning and review decisions; the LLM does not execute code or call external tools. This is a narrower notion of “agentic” than fully autonomous research agents [13], and we state it plainly.

Joint role ablation. The ablation switches planner and reviewer together. The transcript analyses provide observational evidence about each role’s behavior, but disentangling their causal contributions requires a factorial design (LLM planner with analytic reviewer, and vice versa), which we leave to future work.

Literature priors. A mid-size or frontier LLM has read the operator-learning literature and may “know” that FNO suits periodic problems. This is simultaneously a feature (informed priors) and a confound (it is not prior-free discovery); the PSO condition provides the prior-free reference point.

Determinism. LLM planning is stochastic; we report multiple seeds and release transcripts rather than claiming bit-for-bit reproducibility.

Screening fidelity. The swarm screens at reduced fidelity; absolute errors should be read against the converged baselines, not as final accuracies.

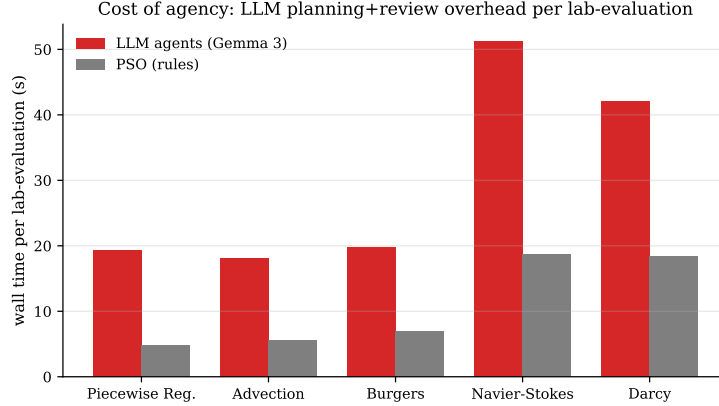


Figure 7: Per lab-evaluation wall time, LLM vs. PSO. The gap is the cost of LLM planning and peer review at identical training budget.

Table 5: Single-operator training time to convergence at screening fidelity (Navier–Stokes), mean over seeds.

Operator (NS-2D, screening fidelity)	Train time (s)
POD-DeepONet	4.1±0.0
FNO h32 m12	9.7±0.4
FNO h64 m12	10.5±0.4
FNO h128 m16	17.1±0.1
Discovered hybrid (NS)	20.3±0.5
Discovered hybrid (Darcy)	22.5±0.2
Pure Transformer	37.4±0.1
DeepONet faithful	48.4±0.7

Model scale. We use a single local model (gemma3:12b); whether a frontier model changes discovery quality is left to future work, enabled by the model-agnostic backend.

10 Reproducibility

Code, the resume-safe runner, exact configurations, and the complete per-run LLM transcripts are available at <https://github.com/luislootx/AI-SC>. Every reported LLM run ships its `llm_transcript.jsonl`; the evidence audit (Table 3) is reproducible directly from these files. The LLM backend (gemma3:12b via Ollama) runs locally with no paid API. Representative planner and reviewer prompts are listed in Appendix A.

11 Conclusion

We presented an agentic AI Science Community that discovers neural operator architectures through LLM-driven planning and peer review over a swarm of virtual labs, with a deterministic numerical worker and a controlled ablation against rule-based coordination. The honest cross-regime finding is that no operator is universal; the contribution is a reproducible, auditable methodology for agentic architecture discovery that recovers regime-appropriate, parameter-lean hybrids at a quantified LLM overhead. Future work includes frontier LLM backends, a factorial planner/reviewer ablation, larger block vocabularies (graph operators, neural Galerkin), action-space debiasing of the planner, and full-fidelity re-training of every discovered architecture.

Acknowledgments and Disclosure of Funding

We thank the organizers of the ICERM Hot Topics Workshop on Agentic Scientific Computing and Scientific Machine Learning, where a preliminary version of this work was presented as a poster, and the Texas A&M ECEN graduate program for travel support.

References

- [1] Ulisses Braga-Neto. The AI scientific community: Agentic virtual lab swarms. *arXiv preprint arXiv:2603.21344*, 2026.
- [2] Shuhao Cao. Choose a transformer: Fourier or galerkin. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 34, 2021.
- [3] Angelica Chen, David Dohan, and David So. EvoPrompting: Language models for code-level neural architecture search. In *Advances in Neural Information Processing Systems 36 (NeurIPS)*, 2023. arXiv:2302.14838.
- [4] Thomas Elsken, Jan Hendrik Metzen, and Frank Hutter. Neural architecture search: A survey. *Journal of Machine Learning Research*, 20(55):1–21, 2019.
- [5] Zhongkai Hao, Zhengyi Wang, Hang Su, Chengyang Ying, Yinpeng Dong, Songming Liu, Ze Cheng, Jian Song, and Jun Zhu. GNOT: A general neural operator transformer for operator learning. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, volume 202, 2023.
- [6] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. MetaGPT: Meta programming for a multi-agent collaborative framework. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2308.00352.
- [7] James Kennedy and Russell Eberhart. Particle swarm optimization. In *Proceedings of the IEEE International Conference on Neural Networks (ICNN’95)*, volume 4, pages 1942–1948, 1995.
- [8] Nikola Kovachki, Zongyi Li, Burigede Liu, Kamyar Azizzadenesheli, Kaushik Bhattacharya, Andrew Stuart, and Anima Anandkumar. Neural operator: Learning maps between function spaces with applications to PDEs. *Journal of Machine Learning Research*, 24(89):1–97, 2023.
- [9] Guohao Li, Hasan Abed Al Kader Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. CAMEL: Communicative agents for “mind” exploration of large language model society. In *Advances in Neural Information Processing Systems 36 (NeurIPS)*, 2023. arXiv:2303.17760.
- [10] Zongyi Li, Daniel Zhengyu Huang, Burigede Liu, and Anima Anandkumar. Fourier neural operator with learned deformations for PDEs on general geometries. *Journal of Machine Learning Research*, 24(388):1–26, 2023. arXiv:2207.05209.
- [11] Zongyi Li, Nikola Kovachki, Kamyar Azizzadenesheli, Burigede Liu, Kaushik Bhattacharya, Andrew Stuart, and Anima Anandkumar. Fourier neural operator for parametric partial differential equations. In *International Conference on Learning Representations (ICLR)*, 2021.
- [12] Hanxiao Liu, Karen Simonyan, and Yiming Yang. DARTS: Differentiable architecture search. In *International Conference on Learning Representations (ICLR)*, 2019. arXiv:1806.09055.
- [13] Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. Towards end-to-end automation of AI research. *Nature*, 651(8107):914–919, 2026.
- [14] Lu Lu, Pengzhan Jin, Guofei Pang, Zhongqiang Zhang, and George Em Karniadakis. Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators. *Nature Machine Intelligence*, 3(3):218–229, 2021.

- [15] Lu Lu, Xuhui Meng, Shengze Cai, Zhiping Mao, Somdatta Goswami, Zhongqiang Zhang, and George Em Karniadakis. A comprehensive and fair comparison of two neural operators (with practical extensions) based on FAIR data. *Computer Methods in Applied Mechanics and Engineering*, 393:114778, 2022.
- [16] Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, Juyuan Xu, Dahai Li, Zhiyuan Liu, and Maosong Sun. ChatDev: Communicative agents for software development. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024. arXiv:2307.07924.
- [17] Md Ashiqur Rahman, Zachary E. Ross, and Kamyar Azizzadenesheli. U-NO: U-shaped neural operators. *Transactions on Machine Learning Research (TMLR)*, 2023. arXiv:2204.11127.
- [18] Maziar Raissi, Paris Perdikaris, and George Em Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707, 2019.
- [19] Esteban Real, Alok Aggarwal, Yanping Huang, and Quoc V. Le. Regularized evolution for image classifier architecture search. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 33, pages 4780–4789, 2019.
- [20] Esteban Real, Chen Liang, David R. So, and Quoc V. Le. AutoML-Zero: Evolving machine learning algorithms from scratch. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, volume 119 of *Proceedings of Machine Learning Research*, pages 8007–8019, 2020. arXiv:2003.03384.
- [21] Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M. Pawan Kumar, Emilien Dupont, Francisco J. R. Ruiz, Jordan S. Ellenberg, Pengming Wang, Omar Fawzi, Pushmeet Kohli, and Alhussein Fawzi. Mathematical discoveries from program search with large language models. *Nature*, 625(7995):468–475, 2024.
- [22] Makoto Takamoto, Timothy Praditia, Raphael Leiteritz, Dan MacKinlay, Francesco Alesiani, Dirk Pflüger, and Mathias Niepert. PDEBench: An extensive benchmark for scientific machine learning. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, volume 35, 2022.
- [23] Juan Diego Toscano, Daniel T. Chen, and George Em Karniadakis. ATHENA: Agentic team for hierarchical evolutionary numerical algorithms. *arXiv preprint arXiv:2512.03476*, 2025.
- [24] Tapas Tripura and Souvik Chakraborty. Wavelet neural operator for solving parametric partial differential equations in computational mechanics problems. *Computer Methods in Applied Mechanics and Engineering*, 404:115783, 2023.
- [25] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Shaokun Zhang, Erkang Zhu, Beibin Li, Li Jiang, Xiaoyun Zhang, and Chi Wang. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. In *First Conference on Language Modeling (COLM)*, 2024. arXiv:2308.08155.
- [26] Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V. Le, Denny Zhou, and Xinyun Chen. Large language models as optimizers. In *International Conference on Learning Representations (ICLR)*, 2024.
- [27] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-judge with MT-Bench and chatbot arena. In *Advances in Neural Information Processing Systems 36 (NeurIPS), Datasets and Benchmarks Track*, 2023. arXiv:2306.05685.
- [28] Mingkai Zheng, Xiu Su, Shan You, Fei Wang, Chen Qian, Chang Xu, and Samuel Albanie. Can GPT-4 perform neural architecture search? *arXiv preprint arXiv:2304.10970*, 2023.
- [29] Barret Zoph and Quoc V. Le. Neural architecture search with reinforcement learning. In *International Conference on Learning Representations (ICLR)*, 2017.

A Agent Prompts

The planner and reviewer system prompts are reproduced verbatim below (PDE name instantiated per run); each user prompt carries the lab’s current genome, its recent fitness records, the global best, and an anonymized summary of peer labs, exactly as recorded in every run’s `llm_transcript.jsonl`.

Planner system prompt.

You are the planning agent of a virtual research lab in a swarm of labs collectively discovering neural operator architectures for 2D Navier-Stokes. Each lab is one PSO particle; your job is to propose the next architecture (genome) for your lab to train and evaluate next iteration.

You must balance EXPLORATION (novel hybrids, unexplored block combinations) with EXPLOITATION (move toward the global best architecture observed in the swarm). Use the lab’s current `exploration_rate` as a guide: high $\rightarrow$ favor exploration, low $\rightarrow$ favor exploitation. Consider what other labs are doing -- if the swarm is converging, propose something different to maintain diversity and avoid groupthink.

Output STRICT JSON ONLY (no prose, no fences) matching this schema:

```
{
  "action": "explore" | "exploit" | "hybridize",
  "rationale": "<1-2 sentences explaining your choice>",
  "new_genome": { "block_sequence": [...], "hidden_channels": <int>,
    "fourier_modes": <int>, "activation": "<str>", "use_gating": <bool>,
    "use_skip_connections": <bool>, "dropout_rate": <float>,
    "learning_rate": <float>, "weight_decay": <float> }
}
- ALLOWED_BLOCKS = ["fourier", "attention", "branch_trunk", "wavelet",
  "residual_conv"]
```

Reviewer system prompt.

You are an anonymous peer reviewer evaluating neural operator architectures proposed by other virtual labs in a research swarm.

You must rank the candidates on overall scientific merit for solving 2D Navier-Stokes operator learning. Reward: high accuracy, good generalization under noise, parameter efficiency, architectural novelty (unusual block combinations), and theoretical soundness (e.g. fourier blocks for periodic domains, attention for nonlocal coupling). Penalize: groupthink (identical to global best), pathological setups (too few or too many blocks, mismatched modes), or poor measured fitness.

Output STRICT JSON ONLY:

```
{
  "scores": { "<lab_id>": <float in [0,1]>, ... },
  "rationale": "<1-2 sentences total, not per lab>"
}
```

Scores must sum to roughly 1.0 across the candidates (soft-vote distribution).

B Per-seed Results

Table 6 lists the discovered block sequence, screening relative L_2 , and parameter count for every run of the campaign (5 PDEs $\times$ 2 conditions $\times$ 3 seeds). Machine-readable versions are generated into `results/aggregate_v2.json` in the repository.

C Experimental Details

Table 7 lists the screening-phase configuration per PDE (from each run’s released `config.json`). Both conditions share these budgets exactly; only the planner/reviewer policy differs. Every candidate

Table 6: Per-seed discovered architectures, all 30 runs. Blocks abbreviated F=fourier, A=attention, W=wavelet, R=residual_conv, T=branch_trunk.

PDE	Cond.	Seed	Discovered blocks	rel. L_2	Params
Piecewise Reg. (1D)	LLM	42	A-W-R-F	0.0429	692K
Piecewise Reg. (1D)	LLM	137	R-R-W-T-A	0.0419	122K
Piecewise Reg. (1D)	LLM	2024	A-R-W-T	0.0404	647K
Piecewise Reg. (1D)	PSO	42	W-A-W-W	0.0410	333K
Piecewise Reg. (1D)	PSO	137	W-R-W-W-W	0.0396	128K
Piecewise Reg. (1D)	PSO	2024	R-W-W-R-W	0.0410	134K
Advection (1D)	LLM	42	F-A-W-T	0.0191	143K
Advection (1D)	LLM	137	A-R-F-F-T	0.0105	494K
Advection (1D)	LLM	2024	F-A-W-T	0.0180	182K
Advection (1D)	PSO	42	F-A-T-T-T-T	0.0186	559K
Advection (1D)	PSO	137	A-F-W-F	0.0173	576K
Advection (1D)	PSO	2024	R-F-F	0.0189	231K
Burgers (1D)	LLM	42	F-W-T-F-R-A	0.0759	1.1M
Burgers (1D)	LLM	137	F-W-A-R	0.0745	142K
Burgers (1D)	LLM	2024	F-A-T-W	0.0894	217K
Burgers (1D)	PSO	42	F-R-F-T-A-F	0.0767	335K
Burgers (1D)	PSO	137	F-F-F-T-A	0.0707	87K
Burgers (1D)	PSO	2024	F-R-F	0.0653	169K
Navier-Stokes (2D)	LLM	42	F-A-T-W-F	0.0011	1.4M
Navier-Stokes (2D)	LLM	137	F-A-W-T-R-F	0.0003	2.6M
Navier-Stokes (2D)	LLM	2024	A-W-T-A-F	0.0015	2.8M
Navier-Stokes (2D)	PSO	42	F-F-F-F	0.0007	10.9M
Navier-Stokes (2D)	PSO	137	F-W-T-F-F	0.0006	921K
Navier-Stokes (2D)	PSO	2024	F-F-F-F	0.0007	5.1M
Darcy (2D)	LLM	42	R-A-W-F	0.0099	1.9M
Darcy (2D)	LLM	137	A-W-R-F-W	0.0091	1.3M
Darcy (2D)	LLM	2024	A-W-R-T-F	0.0095	930K
Darcy (2D)	PSO	42	W-W-R-R-F	0.0089	1.7M
Darcy (2D)	PSO	137	W-W-F-W	0.0082	933K
Darcy (2D)	PSO	2024	W-W-W-W	0.0116	257K

is trained from scratch for the stated epochs each iteration; robustness (noisy relative L_2) is measured under additive input noise $\varepsilon \sim \mathcal{N}(0, 0.1^2)$. The swarm uses $N = 16$ labs for $T = 20$ iterations in all runs. The LLM backend is gemma3:12b served by Ollama with an 8192-token context window; all timings are on a single RTX 4080. Converged-baseline budgets (Table 1) are per-family—FNO variants 50 epochs, hybrids and POD-DeepONet and the pure transformer 80, DeepONet-faithful 600 with input/output normalization—as encoded in the released `validate_baselines_v3.py`.

Table 7: Screening-phase configuration per PDE (identical across conditions and seeds).

PDE	Dim	Resolution	Train / test	Epochs per candidate
Piecewise Reg.	1D	128	256 / 64	40
Advection	1D	128	256 / 64	40
Burgers	1D	128	256 / 64	40
Navier–Stokes	2D	32×32	512 / 128	50
Darcy	2D	32×32	512 / 128	50